Práctica 3.1 — Entrenar y servir un bot sencillo

Objetivo: entrenar un agente de aprendizaje por refuerzo (PPO, Stable-Baselines3) en un entorno simulado (`CartPole-v1`, sustituto didáctico de la mecánica de equilibrio de una nave) y medir el costo y la latencia de su inferencia.

Archivos:

- `ai/training/entrenar_bot.py`: entrena, evalúa, mide la inferencia y guarda el modelo y un JSON con los resultados.
- `ai/training/practica_3_1.ipynb`: el mismo script listo para Google Colab. Corre CPU y GPU y arma la tabla comparativa.
- `ai/training/resultados/colab/`: modelos y mediciones de Colab (CPU y GPU T4).
- `ai/training/resultados/laptop/`: modelo y medición de la laptop (solo CPU).

Cómo reproducir

En Colab con GPU T4: sube `practica_3_1.ipynb` a colab.research.google.com, elige *Entorno de ejecución → Cambiar tipo de entorno → GPU T4* y ejecuta todo. La última celda descarga `resultados.zip`.

Local (CPU):

```
python -m venv .venv
.venv/Scripts/python -m pip install torch --index-url https://download.pytorch.org/whl/cpu
.venv/Scripts/python -m pip install gymnasium stable-baselines3
cd ai/training
../../.venv/Scripts/python entrenar_bot.py --device cpu
```

Resultados

Mismo código, 100,000 timesteps, `seed=0`. La latencia de inferencia se mide siempre en CPU con **un solo hilo** (`torch.set_num_threads(1)`), porque lo que interesa es cuántas decisiones da un núcleo. Se promedian 10,000 llamadas a `predict` después de 200 de calentamiento.

Medición	Colab, entrenamiento en CPU	Colab, entrenamiento en GPU T4	Laptop, entrenamiento en CPU
Equipo	2 vCPU x86_64, Tesla T4, torch 2.11.0+cu130	(misma VM)	AMD Ryzen (Family 25 Model 68), 12 hilos, Windows 11, torch 2.15.0+cpu
Tiempo de entrenamiento	**132.6 s**	**222.9 s**	173.4 s
Recompensa media (10 episodios, máx. 500)	500.0	500.0	500.0
Latencia de inferencia (1 hilo de CPU)	0.359 ms	0.432 ms	0.224 ms
Decisiones/s por núcleo	2,787	2,315	4,460

Fechas (UTC): Colab 2026-11-05 01:39 y 01:43; laptop 2026-11-05 01:17. Datos en `ai/training/resultados/*/resultados_*.json`.

¿CPU o GPU? La GPU fue 68 % más lenta

En la misma VM de Colab, entrenar en la **GPU T4 tardó 222.9 s** y en **CPU 132.6 s**: la GPU fue 1.68 veces más lenta. Por qué:

- La red de PPO es diminuta: un MLP de 2 capas de 64 neuronas. Un paso de la red cuesta microsegundos en cualquier hardware.
- PPO con un solo entorno pasa casi todo el tiempo simulando CartPole, que corre en CPU. En cada paso hay que copiar la observación a la GPU y traer la acción de vuelta. Esas copias cuestan más que el cálculo que se ahorra.
- La GPU solo gana cuando hay mucho cálculo en paralelo: redes grandes, entradas de imagen o cientos de entornos vectorizados. Nada de eso pasa en CartPole.

Conclusión: para este bot, **entrenar en CPU es más rápido y más barato**. La GPU solo se justificaría con un modelo más grande, por ejemplo un bot que vea la arena como imagen.

Las dos latencias de inferencia de Colab (0.359 y 0.432 ms) se midieron las dos en CPU con modelos del mismo tamaño. La diferencia de 20 % es ruido de una vCPU compartida, no efecto de dónde se entrenó el modelo.

Dimensionamiento: 2,000 bots a 10 Hz

```
decisiones/s = bots activos × frecuencia = 2,000 × 10 = 20,000 inferencias/s
```

Capacidad medida por núcleo	vCPU sin margen	Con 30 % de margen
Colab, peor medición: 2,315/s	8.64	**12 vCPU**
Colab, mejor medición: 2,787/s	7.18	10 vCPU
Laptop: 4,460/s	4.48	6 vCPU

Para dimensionar la nube se usa la medición de **vCPU de nube** (Colab), no la de la laptop. Un núcleo de escritorio rinde 1.6 a 1.9 veces más que una vCPU compartida. Con la peor medición, que es lo conservador, se necesitan **12 vCPU**, muy cerca de las 11 del ejemplo del material (2,500/s por vCPU). Antes de producción conviene repetir la medición en el tipo de máquina real (por ejemplo `c3-standard-4`), porque Colab no garantiza qué CPU asigna.

Costo estimado por hora

Precios **oficiales** de la página de precios de Compute Engine (cloud.google.com/products/compute/pricing/general-purpose, la misma fuente que usa la calculadora), región `us-central1` **(Iowa)**, en **USD**, consultados el **2026-11-09**. Solo incluyen la máquina virtual (vCPU + memoria, Linux); no incluyen disco, red ni IP pública, que con este tamaño son pequeños frente al cómputo. Mes = 730 horas.

Máquina (4 vCPU)	Por hora (bajo demanda)	CUD 1 año	CUD 3 años
`c3-standard-4` (16 GiB)	0.201608	0.127016	0.090724
`c3-highcpu-4` (8 GiB)	0.170104	0.107168	0.076548
`e2-highcpu-4` (4 GiB)	0.09894048	0.062332504	0.04452322
`e2-standard-2` (2 vCPU, 8 GiB)	0.06701142	—	—

(CUD = descuento por compromiso de uso de recursos.)

Inferencia: 2,000 bots a 10 Hz = 12 vCPU = 3 máquinas de 4 vCPU, 24/7

Opción	USD/hora	USD/mes	USD por bot y hora
3 × `c3-standard-4`, bajo demanda (la del dimensionamiento)	**0.6048**	**441.52**	**0.000302**
3 × `c3-highcpu-4`, bajo demanda (el modelo no necesita 16 GiB)	0.5103	372.53	0.000255
3 × `c3-standard-4` con CUD de 1 año	0.3810	278.17	0.000191
3 × `c3-standard-4` con CUD de 3 años	0.2722	198.69	0.000136
3 × `e2-highcpu-4`, bajo demanda	0.2968	216.68	0.000148

Lectura:

- El costo base es **USD 0.60 por hora** (≈ USD 442 al mes) para tener 2,000 bots decidiendo 10 veces por segundo, o sea **USD 0.0003 por bot y hora**.

- `c3-highcpu-4` ahorra 16 % sin perder vCPU: la red del bot pesa unos KB, la memoria extra de `standard` no se usa.
- `e2` es la más barata, pero sus vCPU son de rendimiento variable (comparten núcleo); la capacidad por vCPU se midió en Colab y habría que repetirla en `e2` antes de elegirla. Para un servicio que debe responder en cada tick, conviene C3.
- Un CUD solo conviene si los bots corren todo el año; para un juego que arranca, primero bajo demanda y autoescalado según los bots activos.

Entrenamiento

Se demostró que basta CPU (la GPU fue 68 % más lenta). Un entrenamiento dura ~2 min en 2 vCPU: en `e2-standard-2` cuesta **USD 0.0022** por entrenamiento (0.06701142 × 2/60), prácticamente nada; también puede hacerse gratis en Colab. Por eso no vale la pena pagar GPU para este bot. El entrenamiento, a diferencia del servidor de partida, sí puede correr en instancias Spot: si lo interrumpen se reanuda desde el último punto guardado.

Pendiente opcional: confirmar el total armando la misma configuración en la calculadora (cloud.google.com/products/calculator) y guardar el enlace o la captura en `docs/evidencias/`.